

Workshop: Implementierung eines SAT-Solvers

1. Datenhaltung der SAT-Instanz

Im Laufe des Workshops soll eine Klasse `SATInstanz` implementiert werden, die eine aussagenlogische Formel in **konjunktiver Normalform (KNF)** repräsentiert.

🔔 Erinnerung: Formeln in KNF

Eine aussagenlogische Formel in **konjunktiver Normalform (KNF)** besteht aus einer oder mehreren **Klauseln**, die untereinander durch ein logisches **UND ($\wedge$)** verknüpft sind.
Innerhalb einer Klausel befinden sich **Literale** (negierte und nicht negierte Variablen), die jeweils durch ein logisches **ODER ($\vee$)** verbunden sind.

Beispiel (KNF-Formel):
 $(A \vee \neg B \vee C) \wedge (C \vee \neg D) \wedge B$

Überlege dir zunächst, welche **Attribute** die Klasse `SATInstanz` besitzen sollte. Wie könnte die Encodierung von **Variablen**, **Literalen** und **Klauseln** gespeichert werden?

🔔 Erinnerung: Encodierung von Variablen und Literalen

Variablen werden in der Reihenfolge ihres ersten Auftretens encodiert, beginnend mit 0.

- Ein **positives Literal** wird als $2 \cdot x + 1$ encodiert
- Ein **negatives Literal** wird als $2 \cdot x + 1$ encodiert

wobei x der Encodierung der zugehörigen Variable entspricht.

Beispiel

Formel:
 $A \neg C B$
 $B \neg A$

Variablencodierung:

- $A \rightarrow 0$
- $C \rightarrow 1$
- $B \rightarrow 2$

Literalencodierung:

- $A \rightarrow 0$
- $\neg A \rightarrow 1$
- $C \rightarrow 2$
- $\neg C \rightarrow 3$
- $B \rightarrow 4$
- $\neg B \rightarrow 5$

Geeignete Datenstrukturen

Bei der Implementierung der Klasse `SATInstanz` ist es wichtig, sich **bewusst Gedanken über die Wahl der verwendeten Datenstrukturen** zu machen.

Je nach Verwendungszweck (z. B. schneller Zugriff über Indizes, effiziente Abbildung von Namen auf interne Repräsentationen oder Vermeidung von Duplikaten) eignen sich unterschiedliche Datenstrukturen unterschiedlich gut. Eine durchdachte Wahl kann den Code nicht nur übersichtlicher, sondern auch deutlich effizienter machen.

Eine kompakte Übersicht über **gängige Datenstrukturen, ihre typischen Einsatzgebiete und Zugriffsszeiten** findest du im folgenden Abschnitt:

[📄 Cheat Sheet- Datenstrukturen](#)

2. Einlesen einer SAT-Instanz aus einer Datei

Implementiere eine **Klassenmethode** `from_file` mit der Signatur

```
from_file(cls, filename)
```

die eine SAT-Instanz aus einer Datei einliest. Der Parameter `filename` ist dabei ein String, der den Namen der Datei enthält.

🔔 Erinnerung: Eingabeformat

In dieser Aufgabe werden die einzelnen Klauseln der KNF-Formel als **Zeilen einer Datei** dargestellt. Negationen werden durch eine **Tilde ($\neg$)** ausgedrückt.

Beispiel:
 $(A \vee \neg B \vee C) \wedge (C \vee \neg D) \wedge B$ ist das gleiche wie:
 $A \neg B C$
 $C \neg D$
 B

💡 Tipp: Sieh dir die Beispielergebnisse vorher einmal an, um mit dem Eingabeformat vertraut zu werden.

Die Methode sollte am Ende

1. die Datei öffnen,
2. alle Klauseln einlesen,
3. Variablen intern encodieren,
4. und eine vollständig initialisierte `SATInstanz` zurückgeben können.

3. Ausgabemethoden

Implementiere Methoden, mit denen die interne Repräsentation der SAT-Instanz wieder **lesbar** ausgegeben werden kann:

- `literal_zu_string(literal)`
→ gibt ein einzelnes Literal als String zurück (z. B. "A" oder "¬C")
- `klausel_zu_string(klausel)`
→ gibt eine gesamte Klausel als String aus (z. B. "A ¬C D")
- `belegung_zu_string(belegung, ...)`
→ gibt eine vollständige oder teilweise Variablenbelegung in lesbarer Form aus

Diese Methoden sollen vor allem beim **Debuggen** und bei der **Ausgabe gefundener Lösungen** helfen.

💡 Tipp: Überlege dir, welche weiteren Parameter die Methode `belegung_zu_string` noch haben könnte. Mache dir insbesondere Gedanken über **große Eingaben mit vielen Variablen**:

- Reicht es z.B., nur die Variablen auszugeben, die in der Belegung den Wert **wahr** haben?
- Wann könnte es sinnvoll sein, nur Variablen mit einem bestimmten Präfix zu betrachten?

4. Implementierung eines SAT-Solvers

Erweitere die Klasse so, dass sie das **SAT-Problem lösen** kann. Der Solver soll **alle erfüllenden Belegungen** finden zurückgeben können.

💡 Tipp: Überlege dir, welche **Datenstruktur** zur Rückgabe der Lösungen geeignet ist. Mache dir dabei insbesondere Gedanken über Formeln, die viele erfüllende Belegungen haben.

🔔 Erinnerung: Watchlist

Eine **Watchlist** ist eine Datenstruktur, die für jedes Literal eine Menge von Klauseln speichert, die dieses Literal „beobachten“.
Wenn ein Literal durch eine Belegung `false` wird, müssen nur die betroffenen Klauseln überprüft werden, anstatt alle Klauseln zu durchsuchen.
So lassen sich Konflikte frühzeitig erkennen und die Suche effizienter gestalten.

🔔 Erinnerung: Watchlist-basierter Algorithmus

Unser Algorithmus arbeitet in etwa so:

1. **Variablenwahl**: Wähle die nächste nicht belegte Variable aus.
2. **Belegung ausprobieren**: Setze sie auf `wahr` oder `false`.
3. **Watchlist prüfen**: Das Literal, das durch die neu hinzugekommene Belegung falsch ist, kann nicht mehr zur Erfüllung einer Klausel dienen. Die Klausel muss ein anderes ihrer Literale beobachten, das `None` oder `wahr` ist.
 - Wenn eine Klausel nicht mehr erfüllbar ist, entsteht ein Konflikt. Breche diesen Zweig ab (Backtracking).
4. **Rekursiver Aufruf**: Ansonsten fahre mit der nächsten Variablen fort.
5. **Lösung gefunden**: Wenn alle Variablen belegt sind und keine Klausel falsifiziert ist, gib die Belegung als Lösung aus.

👉 Durch **Backtracking** werden automatisch alle möglichen Belegungen geprüft.

5. Zusatz

Erweiterung

Entwickle und implementiere einen Algorithmus, der aussagenlogische Formeln beliebiger Form in die konjunktive Normalform überführt.

Verbesserung

Überlege welche Möglichkeiten es gibt, den Watchlist-basierten SAT-Algorithmus **effizienter** zu gestalten:

💡 Tipp: Überlege dir, welchen Einfluss die Reihenfolge der Variablen bei der Variablenzuweisung (Schritt 1) auf die Effizienz des Algorithmus' hat. Führe dir diesen Einfluss mit Hilfe der Eingabe

$A B$
 $\neg A C$
 $\neg B C$
 $\neg C D$
 $\neg D E$
 D

vor Augen.

GUI

Implementiere ein **Interface mit Tkinter**, in dem man ad-hoc eine Formel eingeben, diese um Klauseln erweitern und sich die Lösungen grafisch anzeigen lassen kann.

Cheat Sheet

Dieses Cheat Sheet fasst nützliche Datenstrukturen, Operationen und Python-Konzepte zusammen, die beim Implementieren deines SAT-Solvers hilfreich sein könnten.

Datenstrukturen

Datenstruktur	Typische Anwendung
<code>list</code>	Geordnete Sammlung von Elementen
<code>tuple</code>	Unveränderliche Sammlung von Elementen, keine Änderung möglich
<code>set</code>	Eindeutige Elemente
<code>dict</code>	Schlüssel → Wert Abbildungen
<code>generator</code>	Speicherfreundliche Iteration über große Mengen

Listenoperationen

Operation	Beschreibung	Laufzeit
<code>lst.append(x)</code>	Element ans Ende anhängen	$O(1)$ amortisiert
<code>lst.pop()</code>	Letztes Element entfernen und zurückgeben	$O(1)$
<code>lst.pop(i)</code>	Element an Index <code>i</code> entfernen	$O(n)$
<code>x in lst</code>	Mitgliedschaft prüfen	$O(n)$
<code>lst.sort()</code>	Liste in-place sortieren	$O(n \log n)$
<code>sorted(lst)</code>	Neue sortierte Liste erzeugen	$O(n \log n)$
<code>list(zip(lst1, lst2))</code>	Paare aus zwei Listen bilden (endet bei kürzerer Liste)	$O(\min(n, m))$
List Comprehension [<code><Ausdruck></code> for <code><Variable></code> in <code><Iterable></code> if <code><Bedingung></code>]	Neue Liste mit Filter/Transformation erzeugen	$O(n)$

Tuple-Operationen

Operation	Beschreibung	Laufzeit
<code>t = (a, b, c)</code>	Tuple erzeugen	$O(n)$
<code>t[i]</code>	Zugriff auf Element an Index <code>i</code>	$O(1)$
<code>x in t</code>	Mitgliedschaft prüfen	$O(n)$
<code>len(t)</code>	Länge des Tuples bestimmen	$O(1)$
<code>for x in t</code>	Iteration über Tuple	$O(n)$
<code>tuple(iterable)</code>	Tuple aus Iterable erzeugen	$O(n)$

Set-Operationen

Operation	Beschreibung	Laufzeit
<code>s.add(x)</code>	Element zum Set hinzufügen	$O(1)$
<code>s.remove(x)</code>	Element entfernen (Fehler, falls nicht vorhanden)	$O(1)$
<code>s.discard(x)</code>	Element entfernen (kein Fehler, falls nicht vorhanden)	$O(1)$
<code>x in s</code>	Mitgliedschaft prüfen	$O(1)$
<code>s1 s2</code>	Vereinigung zweier Sets	$O(\text{len}(s1) + \text{len}(s2))$
<code>s1 & s2</code>	Schnittmenge zweier Sets	$O(\min(\text{len}(s1), \text{len}(s2)))$
<code>s1 - s2</code>	Differenz zweier Sets	$O(\text{len}(s1))$
<code>set(iterable)</code>	Erzeugt Set aus Iterable (Duplikate werden entfernt)	$O(n)$

String-Operationen

Operation	Beschreibung	Laufzeit
<code>s.strip()</code>	Entfernt führende und nachfolgende Whitespaces	$O(n)$
<code>s.split()</code>	Tellt String an Whitespaces in Liste	$O(n)$
<code>sep.join(lst)</code>	Verbindet Liste von Strings zu einem String	$O(n)$
<code>s.startswith(prefix)</code>	Prüft, ob String mit <code>prefix</code> beginnt	$O(\text{len}(\text{prefix}))$
<code>s.endswith(suffix)</code>	Prüft, ob String mit <code>suffix</code> endet	$O(\text{len}(\text{suffix}))$
<code>s.replace(a, b)</code>	Ersetzt Teilstrings	$O(n)$
<code>len(s)</code>	Länge des Strings bestimmen	$O(1)$
<code>s[i]</code>	Zugriff auf Zeichen an Index <code>i</code>	$O(1)$
<code>for c in s</code>	Iteration über String	$O(n)$

Dict-Operationen

Operation	Beschreibung	Laufzeit
<code>d[key] = value</code>	Setzt oder überschreibt den Wert für <code>key</code>	$O(1)$
<code>value = d[key]</code>	Zugriff auf den Wert zu <code>key</code> (<code>KeyError</code> , falls nicht vorhanden)	$O(1)$
<code>value = d.get(key, default)</code>	Zugriff mit Default-Wert	$O(1)$
<code>key in d</code>	Prüft, ob ein Schlüssel existiert	$O(1)$
<code>del d[key]</code>	Löscht Schlüssel und Wert	$O(1)$
<code>d.keys()</code>	Gibt alle Schlüssel zurück	$O(n)$
<code>d.values()</code>	Gibt alle Werte zurück	$O(n)$
<code>d.items()</code>	Gibt Schlüssel-Wert-Paare zurück	$O(n)$
<code>dict(iterable)</code>	Erzeugt ein Dict aus den Schlüssel-Werte-Paaren eines Iterables	$O(n)$

Generatoren (yield)

Konzept / Operation	Beschreibung	Laufzeit
<code>yield x</code>	Gibt einen Wert zurück und pausiert die Funktion	$O(1)$
<code>next(gen)</code>	Liefert das nächste Element des Generators	$O(1)$ pro Element
<code>for x in gen</code>	Iteriert über alle erzeugten Werte	$O(n)$ für n Elemente
Generator-Funktion	Funktion mit mindestens einem <code>yield</code>	$O(1)$ für jeden erzeugten Wert
Generator-Ausdruck (<code>x for x in iterable</code>)	Erzeugt Werte on-demand aus einem Iterable	$O(1)$ pro Element

Bitoperationen

Operation	Beschreibung	Laufzeit
<code>x << n</code>	Bitshift nach links (Multiplikation mit 2^n)	$O(1)$
<code>x >> n</code>	Bitshift nach rechts (Ganzzahldivision durch 2^n)	$O(1)$
<code>x & y</code>	Bitweises UND	$O(1)$
<code>x y</code>	Bitweises ODER	$O(1)$
<code>x ^ y</code>	Bitweises XOR	$O(1)$
<code>~x</code>	Bitweises NICHT (NOT)	$O(1)$

💡 Tipp: Sieh dir nochmal an, wie man Operationen zwischen Literalen und Variablen mit Hilfe der sehr effizienten Bitoperationen umsetzen kann.

Beispiel: `pos = 2*var_index, neg = 2*var_index+1`

Klassenmethoden (@classmethod)

- Wird mit `@classmethod` dekoriert.
- Enthält als ersten Parameter die **Klasse selbst** (`cls`) statt einer Instanz (`self`).
- Sinnvoll, um alternative Konstruktoren zu implementieren (z. B. aus einer Datei).

```
class SATInstanz:
    @classmethod
    def from_file(cls, filename):
        instanz = cls()
        # Initialisierung aus Datei
        return instanz
```

7. Link-Sammlung

Ich will **wiederholen** ...

- ... was eine **aussagenlogische Formel** ist.
- ... was die **konjunktive Normalform (KNF)** ist.
- ... was das **SAT-Problem** ist.
- ... wie man einen **einfachen SAT-Solver in Python** baut.

Ich will **lernen** ...

- ... wie man **eine aussagenlogische Formel in die konjunktive Normalform** bringt.
- ... was **NP-schwer** bedeutet.
- ... wie man **SAT auf 3-SAT reduziert**.
- ... wie man **3-SAT auf 3-COL reduziert**.

Ich **interessiere** mich besonders für ...

- ... **Aussagenlogik**.
- ... **Komplexitätstheorie**.